

BÁO CÁO KẾT QUẢ THỬ NGHIỆM

Thời gian thực hiện: 01/10 – 10/10/2026

Sinh viên thực hiện: Đỗ Bá Luân - 23520895

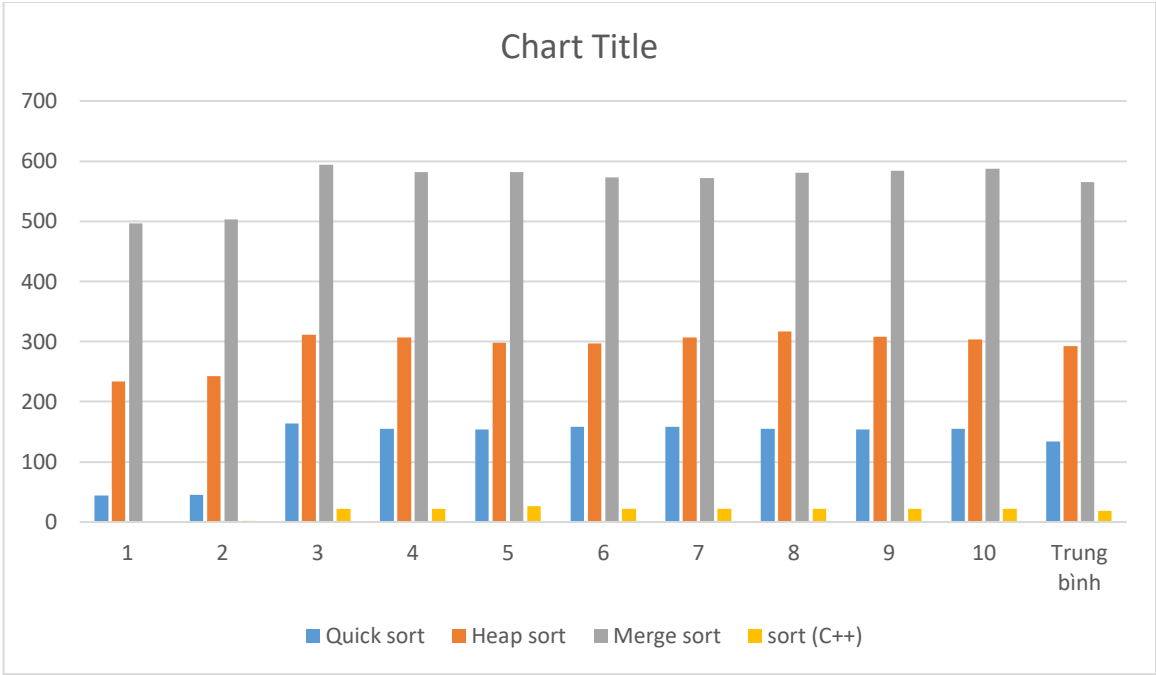
Nội dung báo cáo:

I. Kết quả thử nghiệm

1. Bảng thời gian thực hiện¹

Dữ liệu	Thời gian thực hiện (ms)			
	Quick sort	Heap sort	Merge sort	sort (C++)
1	43.79	233.48	496.66	0.95
2	45.05	243.06	503.02	1.64
3	163.40	311.80	593.88	22.12
4	155.27	307.05	581.69	21.47
5	153.76	297.82	581.58	26.12
6	157.83	297.29	572.35	21.45
7	157.74	306.64	571.38	21.62
8	155.17	316.77	580.70	21.43
9	153.88	308.04	583.94	22.14
10	155.10	303.16	587.53	21.78
Trung bình	134.10	292.51	565.27	18.07

2. Biểu đồ (cột) thời gian thực hiện



II. Kết luận:

Qua quá trình thực nghiệm và so sánh thời gian chạy của các thuật toán sắp xếp trên 10 bộ dữ liệu số thực (mỗi bộ gồm 1 triệu phần tử), em rút ra được một số kết luận như sau:

Thứ nhất, hàm sort sẵn có của C++ cho tốc độ xử lý vượt trội hơn hẳn so với các thuật toán còn lại trong mọi trường hợp. Với bộ dữ liệu đã được sắp xếp sẵn như dãy 1 và dãy 2, hàm này hoàn thành chỉ trong hơn 1 mili-giây. Đối với các dãy ngẫu nhiên, thời gian thực thi cũng chỉ dao động khoảng 21 đến 26 mili-giây. Lý do là vì hàm sort trong C++ đã được tối ưu hóa rất tốt ở cấp độ thư viện và hệ thống.

Thứ hai, đối với ba thuật toán tự cài đặt:

- QuickSort có tốc độ chạy khá tốt và tương đối ổn định. Do nhóm em chọn phần tử chốt (pivot) ở giữa mảng nên đối với dãy số đã tăng hoặc giảm sẵn, thuật toán phân chia rất đều và đạt thời gian chạy rất nhanh (khoảng 43 đến 45 mili-giây). Đối với các dãy ngẫu nhiên, thời gian thực thi trung bình khoảng 155 mili-giây.
- HeapSort cho thời gian chạy trung bình khoảng 290 đến 310 mili-giây. Mặc dù độ phức tạp lý thuyết là $O(N\log N)$ trong mọi trường hợp, nhưng việc liên tục truy cập các phần tử không liên tiếp trên bộ nhớ khi vun đống khiến tốc độ thực tế bị chậm hơn so với QuickSort.
- MergeSort có thời gian chạy chậm nhất trong 4 phương pháp, trung bình khoảng 560 đến 590 mili-giây. Nguyên nhân chính là do hàm gộp cần phải cấp phát và giải phóng bộ nhớ phụ liên tục cho các mảng con trong quá trình đệ quy, dẫn đến chi phí quản lý bộ nhớ tăng cao.

Tóm lại, thông qua bài tập thực nghiệm này, em không chỉ hiểu rõ hơn cách cài đặt từng thuật toán sắp xếp cơ bản mà còn thấy rõ sự khác biệt giữa độ phức tạp lý thuyết và hiệu năng thực tế khi chạy trên dữ liệu lớn. Trong thực tế lập trình, nếu không có yêu cầu đặc thù thì việc sử dụng hàm sắp xếp chuẩn của ngôn ngữ vẫn là lựa chọn tối ưu nhất.

III. Thông tin chi tiết

Link Github: https://github.com/LuanDoo2005/23520895_IT003.R17.git

1. Báo cáo (tên file: *Sorting_Report_23520895.pdf*)
2. Mã nguồn (tên file: *23520895.cpp*)
3. Dữ liệu thử nghiệm (tên file: *test1.txt* đến *test10.txt*)